

# KINGSGATE INFRASTRUCTURE & OPERATIONS RUNBOOK

---

## Table of Contents

1. Architecture Overview & Module Descriptions
  2. Frontend Dashboard Deployment
  3. Frontend Payment Page Deployment
  4. Integration Script Deployment
  5. Backend API Deployment
  6. Static Assets Management
  7. Certbot / TLS Certificate Renewal
  8. Environment Variables
  9. Route 53 DNS Configuration
  10. ACM (AWS Certificate Manager)
  11. RDS PostgreSQL
  12. Common Troubleshooting
- 

## 1. Architecture Overview & Module Descriptions

**DOMAIN:** kg91.pro (registered via Namecheap, DNS managed via Route 53)

### MODULE 1: Dashboard Application

| Property          | Value                                                    |
|-------------------|----------------------------------------------------------|
| <b>Purpose</b>    | Primary web application dashboard for Kingsgate platform |
| <b>Tech Stack</b> | React + Vite                                             |
| <b>Domain</b>     | kg91.pro (apex domain)                                   |

| Property             | Value                    |
|----------------------|--------------------------|
| <b>S3 Bucket</b>     | kingsgate-india.kg91.pro |
| <b>CloudFront ID</b> | E165LTPDW94GID           |
| <b>ACM Cert</b>      | kg91.pro                 |
| <b>Build Output</b>  | ./dist (Vite default)    |

#### Description:

The main user-facing dashboard application. Built with React and Vite, it provides the core interface for merchants, members, agents, admin etc to interact with the Kingsgate platform. Deployed as a static single-page application (SPA) on S3, served globally via CloudFront for low latency.

## MODULE 2: Payment Page

| Property             | Value                                                                   |
|----------------------|-------------------------------------------------------------------------|
| <b>Purpose</b>       | Standalone payment interface for member channels and 3rd party gateways |
| <b>Tech Stack</b>    | React + Vite                                                            |
| <b>Domain</b>        | kingsgate-payments.kg91.pro                                             |
| <b>S3 Bucket</b>     | www.kingsgate-payments.com                                              |
| <b>CloudFront ID</b> | EK7UEPOT9KO85                                                           |
| <b>ACM Cert</b>      | kingsgate-payments.kg91.pro                                             |
| <b>Build Output</b>  | ./dist (Vite default)                                                   |

#### Description:

Dedicated payment gateway interface. This is the page where the member channels payment interface is hosted and also the 3rd party PGs are hosted under an iframe.

## MODULE 3: Integration Script (SDK)

| Property        | Value                                                                             |
|-----------------|-----------------------------------------------------------------------------------|
| <b>Purpose</b>  | JavaScript SDK for embedding payment functionality on merchant sites via SDK mode |
| <b>Filename</b> | integration.js                                                                    |

| Property          | Value                                                                                                                                                                             |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Location</b>   | S3 bucket - www.kingsgate-payments.com (root level)                                                                                                                               |
| <b>Access URL</b> | <a href="https://s3.ap-southeast-1.amazonaws.com/www.kingsgate-payments.com/integration.js">https://s3.ap-southeast-1.amazonaws.com/www.kingsgate-payments.com/integration.js</a> |

### Description:

A standalone JavaScript file that merchant websites needing SDK mode integration can load via:

html

```
<script src="https://s3.ap-southeast-1.amazonaws.com/www.kingsgate-payments.com/integration.js"></script>
```

This script initializes the Kingsgate payment widget on merchant sites, allowing them to accept payments without building their own payment UI. The file is stored separately in the root of the [www.kingsgate-payments.com](https://www.kingsgate-payments.com) S3 bucket.

**Deployment:** Upload manually to S3 root

## MODULE 4: Backend API (NestJS)

| Property              | Value                                                   |
|-----------------------|---------------------------------------------------------|
| <b>Purpose</b>        | Core application backend, REST API and Web Sockets      |
| <b>Tech Stack</b>     | NestJS (Node.js framework)                              |
| <b>Domain</b>         | api.kingsgate-dev.kg91.pro                              |
| <b>Infrastructure</b> | EC2 instance (ID: i-00e7d9bb0607e5cc4)                  |
| <b>Process Mgmt</b>   | PM2 (process name: kingsgate-dev)                       |
| <b>Web Server</b>     | Nginx (reverse proxy)                                   |
| <b>TLS/SSL</b>        | Certbot / Let's Encrypt (manual renewal, 90-day expiry) |
| <b>Database</b>       | RDS PostgreSQL (private subnet)                         |

### Description:

The main application backend, built with NestJS. Handles all business logic, database operations, authentication, web sockets and API endpoints. Runs on a single EC2 instance managed by PM2 for process monitoring and auto-restart. Nginx acts as a reverse proxy, terminating TLS via Certbot certificates and forwarding requests to the NestJS app running on a local port.

**⚠ No Elastic IP is assigned** - DNS points directly to the EC2 instance's public IP (update Route 53 if the instance is stopped/restarted).

## MODULE 5: Static Assets

| Property          | Value                                     |
|-------------------|-------------------------------------------|
| <b>Purpose</b>    | Storage for images (screenshots/receipts) |
| <b>S3 Bucket</b>  | kingsgate-assets                          |
| <b>CloudFront</b> | None (direct S3 access)                   |
| <b>Access</b>     | Public read                               |

## MODULE 6: Database

| Property       | Value                               |
|----------------|-------------------------------------|
| <b>Service</b> | AWS RDS PostgreSQL                  |
| <b>Access</b>  | Publicly accessible via credentials |

### Description:

Managed PostgreSQL database hosted on AWS RDS. Located in a private subnet with a public IP. It has VPC security group rules for connections.

## MODULE 7: DNS

| Property            | Value        |
|---------------------|--------------|
| <b>Registrar</b>    | Namecheap    |
| <b>DNS Provider</b> | AWS Route 53 |
| <b>Hosted Zone</b>  | kg91.pro     |

All DNS records are managed in Route 53. Nameservers from the Route 53 hosted zone are configured at Namecheap to delegate DNS control to AWS.

# 2. Frontend Dashboard Deployment

**Module:** FE Dashboard (Main Application)

| Property      | Value                    |
|---------------|--------------------------|
| Domain        | kg91.pro                 |
| S3 Bucket     | kingsgate-india.kg91.pro |
| CloudFront ID | E165LTPDW94GID           |
| ACM Cert      | kg91.pro                 |

## Step 1: Build Locally

```
cd /path/to/dps-fe

# Install dependencies (if not already installed)
npm install

# Create production build
npm run build

# Output folder: ./dist (Vite default)
```

The `./dist` folder will contain:

- index.html
- assets/ (CSS, JS chunks, images)
- Other static files

## Step 2: Upload to S3 via AWS Console

1. Log into **AWS Console** → **S3**
2. Open bucket: kingsgate-india.kg91.pro
3. **IMPORTANT:** Delete all existing files first
  - Select all objects → Actions → Delete
  - This prevents old/stale files from remaining
4. Click **"Upload"** → **"Add files"**
5. Select **ALL CONTENTS** inside `./dist` folder

- Do NOT upload the `./dist` folder itself
- Upload everything inside it (index.html, assets/, etc.)

6. Click **"Upload"** and wait for completion

## Step 3: Invalidate CloudFront Cache

After uploading to S3, you **MUST** invalidate CloudFront to clear the cache. Otherwise, users will see the old version for up to 24 hours.

1. **AWS Console** → **CloudFront** → **Distributions**
2. Click on distribution ID: E165LTPDW94GID
3. Go to **"Invalidations"** tab
4. Click **"Create Invalidation"**
5. **Object paths:** `/*`
  - This invalidates ALL files - index.html, CSS, JS, images, etc.
6. Click **"Create Invalidation"**
7. Wait for status to change from "In Progress" to "Completed" (~30-60 sec)

### Verify deployment:

Open <https://kg91.pro> in a browser (hard refresh: `Ctrl+Shift+R` or `Cmd+Shift+R` )

## CloudFront Configuration (Reference)

| This is already configured - for reference only

| Property               | Value                                                                                                                                                         |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Distribution ID        | E165LTPDW94GID                                                                                                                                                |
| Origin Domain          | <a href="http://kingsgate-india.kg91.pro.s3-website-ap-southeast-1.amazonaws.com">http://kingsgate-india.kg91.pro.s3-website-ap-southeast-1.amazonaws.com</a> |
| Default Root Object    | index.html                                                                                                                                                    |
| Viewer Protocol Policy | Redirect HTTP to HTTPS                                                                                                                                        |
| Alternate Domain Name  | kg91.pro                                                                                                                                                      |

| Property        | Value                                |
|-----------------|--------------------------------------|
| SSL Certificate | ACM cert for kg91.pro (in us-east-1) |

## 3. Frontend Payment Page Deployment

**Module:** FE Payment Page

| Property      | Value                       |
|---------------|-----------------------------|
| Domain        | kingsgate-payments.kg91.pro |
| S3 Bucket     | www.kingsgate-payments.com  |
| CloudFront ID | EK7UEPOT9KO85               |
| ACM Cert      | kingsgate-payments.kg91.pro |

### Step 1: Build Locally

```
cd /path/to/payment-page-project
npm install
npm run build

# Output: ./dist
```

### Step 2: Upload to S3 via AWS Console

1. **AWS Console** → **S3** → www.kingsgate-payments.com
2. Delete all existing files **EXCEPT integration.js** (see Section 4)
  - Select all except integration.js → Delete
3. Upload contents of `./dist` (index.html, assets/, etc.)

**⚠ WARNING:** Do NOT delete integration.js during this process. It is deployed separately and lives in the bucket root alongside the payment page build files.

### Step 3: Invalidate CloudFront Cache

1. **CloudFront** → **Distributions** → EK7UEPOT9KO85

## 2. Invalidations → Create Invalidation

### 3. Object paths: /\*

4. Create Invalidation and wait for completion

**Verify:** <https://kingsgate-payments.kg91.pro>

## CloudFront Configuration (Reference)

| Property               | Value                                                                                                                                                             |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Distribution ID        | EK7UEPOT9KO85                                                                                                                                                     |
| Origin Domain          | <a href="http://www.kingsgate-payments.com.s3-website-ap-southeast-1.amazonaws.com">http://www.kingsgate-payments.com.s3-website-ap-southeast-1.amazonaws.com</a> |
| Default Root Object    | index.html                                                                                                                                                        |
| Viewer Protocol Policy | Redirect HTTP to HTTPS                                                                                                                                            |
| Alternate Domain Name  | kingsgate-payments.kg91.pro                                                                                                                                       |
| SSL Certificate        | ACM cert for kingsgate-payments.kg91.pro (us-east-1)                                                                                                              |

## 4. Integration Script Deployment

**Module:** Integration Script (SDK)

| Property   | Value                                                                                                                                                                             |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Filename   | integration.js                                                                                                                                                                    |
| S3 Bucket  | www.kingsgate-payments.com (same as payment page)                                                                                                                                 |
| Access URL | <a href="https://s3.ap-southeast-1.amazonaws.com/www.kingsgate-payments.com/integration.js">https://s3.ap-southeast-1.amazonaws.com/www.kingsgate-payments.com/integration.js</a> |

## Purpose & Usage

The integration.js file is a standalone JavaScript SDK that merchant websites embed to load Kingsgate payment functionality via SDK mode. Merchant parties add this to their HTML:

html

```
<script src="https://s3.ap-southeast-1.amazonaws.com/www.kingsgate-payments.com/integration.js"></script>
```

This initializes the Kingsgate payment widget on their site without requiring them to build custom payment UI.

## Deploy / Update integration.js

 The integration.js file is **NOT** part of the payment page build output. It is a separate file that you maintain and upload independently.

### Steps:

1. Make changes to integration.js locally
2. Test the script thoroughly (CORS, API calls, etc.)
3. **AWS Console** → **S3** → www.kingsgate-payments.com
4. Click "**Upload**" → select integration.js
5. Upload to the **ROOT** of the bucket (not inside any folder)

## 5. Backend API Deployment

| Property     | Value                          |
|--------------|--------------------------------|
| Domain       | api.kingsgate-dev.kg91.pro     |
| EC2 Instance | i-00e7d9bb0607e5cc4            |
| PM2 Process  | kingsgate-dev                  |
| Nginx Config | /etc/nginx/conf.d/default.conf |
| TLS/SSL      | Certbot (Let's Encrypt)        |

 **WARNING:** This EC2 instance does NOT have an Elastic IP. If the instance is stopped and restarted, the public IP changes and you must update the Route 53 A record for api.kingsgate-dev.kg91.pro.

### Step 1: SSH into EC2

```
# Get the current public IP from EC2 Console -> Instances -  
> Instance 234  
# Then SSH in:  
  
ssh -i ~/.ssh/your-key.pem root@<EC2_PUBLIC_IP>
```

---

## Step 2: Deploy via git pull (Recommended)

```
# On the EC2 instance:  
cd ~/apps/dps-be  
  
# Pull latest code from main branch  
git pull  
  
# Install/update dependencies  
npm install  
  
# Build NestJS app  
npm run build  
  
# Reload PM2 process (zero-downtime reload)  
pm2 restart kingsgate-dev  
  
# Verify the process is running  
pm2 status  
pm2 logs kingsgate-dev
```

---

## Step 3: Deploy via SCP (Alternative Method)

If git is not available on EC2:

bash

```
# From your LOCAL machine:
```

```
# Build locally first:
cd /path/to/backend-project
npm run build

# SCP the dist folder to EC2:
scp -i ~/.ssh/your-key.pem -r ./dist root@<EC2_IP>:~/apps/dps-be/dist

# If package.json or dependencies changed:
scp -i ~/.ssh/your-key.pem package.json root@<EC2_IP>:~/apps/dps-be/dist
scp -i ~/.ssh/your-key.pem package-lock.json root@<EC2_IP>:~/apps/dps-be/dist

# Then SSH in and complete deployment:
ssh -i ~/.ssh/your-key.pem root@<EC2_IP>
cd ~/apps/dps-be/dist
npm install
pm2 restart kingsgate-dev
pm2 status
```

## PM2 Commands Reference

|                                    |                            |
|------------------------------------|----------------------------|
| pm2 status                         | # List all processes       |
| pm2 restart kingsgate-dev          | # Hard restart             |
| pm2 stop kingsgate-dev             | # Stop the process         |
| pm2 logs kingsgate-dev --lines 100 | # Stream recent logs       |
| pm2 logs kingsgate-dev -f          | # Follow logs in real-time |

## Nginx Configuration

**Config File:** `/etc/nginx/conf.d/default.conf`

```

# HTTP server - Redirect all HTTP traffic to HTTPS
server {
    listen      80;
    listen [::]:80;
    server_name api.kingsgate-dev.kg91.pro www.api.kingsgate-dev.kg91.pro;

    # Redirect all HTTP traffic to HTTPS
    return 301 https://$host$request_uri;
}

# HTTPS server - Handle SSL/TLS traffic
server {
    listen      443 ssl;
    listen [::]:443 ssl;
    server_name api.kingsgate-dev.kg91.pro www.api.kingsgate-dev.kg91.pro;

    # SSL certificate configuration
    ssl_certificate /etc/letsencrypt/live/api.kingsgate-dev.kg91.pro/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.kingsgate-dev.kg91.pro/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    # Main location block
    location / {
        root    /usr/share/nginx/html;
        index  index.html index.htm;
        try_files $uri /index.html;
    }

    # API endpoint
    location /api/ {
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Host $host:$server_port;

```

```

        proxy_set_header X-Forwarded-Server $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwa
rded_for;
        proxy_pass http://localhost:5000/;
    }

    # WebSocket for socket.io
    location /socket.io/ {
        proxy_pass http://localhost:5000;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwa
rded_for;
        proxy_set_header Host $host;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

```

## Enable the site and reload Nginx

```

sudo ln -s /etc/nginx/conf.d/default.conf /etc/nginx/sites-
enabled/
sudo nginx -t
sudo systemctl reload nginx

```

## Verify Deployment

```

# Check PM2 status
pm2 status

# Check Nginx is running
sudo systemctl status nginx

# Test API endpoint
curl https://api.kingsgate-dev.kg91.pro # returns 201

# Check SSL certificate

```

```
curl -vI https://api.kingsgate-dev.kg91.pro 2>&1 | grep "expire date"
```

## 6. Static Assets Management

**Module:** Static Assets

| Property   | Value                   |
|------------|-------------------------|
| S3 Bucket  | kingsgate-assets        |
| CloudFront | None (direct S3 access) |
| Access     | Public read             |

- used by the application for storing and accessing images(receipts/screenshots)

### Direct S3 URLs (if public)

```
https://kingsgate-assets.s3.amazonaws.com/images/receipt1.png
```

## 7. Certbot / TLS Certificate Renewal

| Property    | Value                               |
|-------------|-------------------------------------|
| Domain      | api.kingsgate-dev.kg91.pro          |
| Certificate | Let's Encrypt via Certbot           |
| Expiry      | 90 days                             |
| Renewal     | MANUAL (no auto-renewal configured) |

**IMPORTANT:** Set a calendar reminder to renew 2-3 weeks before expiry.

### Check Current Certificate Status

```
ssh -i ~/.ssh/your-key.pem root@<EC2_IP>  
sudo certbot certificates
```

Output shows:

- Certificate Name: api.kingsgate-dev.kg91.pro

- Expiry Date: 2025-XX-XX
- Days remaining: XX

---

## Renew Certificate (Dry Run First)

```
# Test renewal without making changes:
sudo certbot renew --dry-run
```

This verifies that Certbot can successfully renew the cert without actually doing it. If the dry run fails, fix any issues before attempting real renewal.

---

## Renew Certificate (Actual Renewal)

```
sudo systemctl stop nginx
sudo certbot renew
sudo systemctl start nginx
```

---

# 8. Environment Variables

 **WARNING:** Never commit .env files to git repositories.

- Backend .env is created directly on the EC2 instance
- Frontend .env files are local only and used at build time

## Backend .env (NestJS on EC2)

**Location:** `~/apps/dps-be/.development.env`

### Edit on EC2

```
ssh -i ~/.ssh/your-key.pem root@<EC2_IP>
nano ~/apps/dps-be/.development.env
```

### After editing

```
npm run build
```

```
pm2 restart kingsgate-dev
```

## Full env Template

```
# Basic Project Configuration
PORT=5000
BASE_URL=http://localhost:5000
SECRET_KEY=your_app_secret_key_here

# PhonePe Payment Configuration
PHONEPE_BASE_URL=https://api-preprod.phonepe.com/apis/pg-sandbox/pg/v1/pay
PHONEPE_API_END_POINT=/pg/v1/pay
MERCHANT_ID=MERCHANT_ID_PLACEHOLDER
MERCHANT_USER_ID=MERCHANT_USER_ID_PLACEHOLDER
SALT_INDEX=1
SAMPLE_SALT_KEY=PHONEPE_SALT_KEY_PLACEHOLDER
PHONEPE_STATUS_END_POINT=/pg/v1/status

# PhonePe API URLs
PHONEPE_API_URL_UAT=https://api-preprod.phonepe.com/apis/pg-sandbox
PHONEPE_API_URL_PROD=https://api.phonepe.com/apis/hermes

# Razorpay Payment Configuration
RAZORPAY_KEY_ID=RAZORPAY_KEY_ID_PLACEHOLDER
RAZORPAY_TEST_KEY_SECRET=RAZORPAY_SECRET_PLACEHOLDER

# Encryption Keys
AES_KEY=your_aes_encryption_key_here
ENCRYPTION_SECRET=your_global_encryption_secret_here

# PostgreSQL Database Configuration
PG_HOST=localhost
PG_PORT=5432
```

```

PG_USERNAME=postgres
PG_PASSWORD=postgres_password_here
PG_DATABASE=your_database_name

# Super Admin Credentials (Seed User)
SUPER_ADMIN_EMAIL=admin@example.com
SUPER_ADMIN_PASSWORD=super_admin_password_here

# AWS Configuration
AWS_REGION=ap-southeast-1
AWS_ACCESS_KEY=AWS_ACCESS_KEY_PLACEHOLDER
AWS_SECRET_KEY=AWS_SECRET_KEY_PLACEHOLDER
S3_BUCKET_NAME=your-s3-bucket-name

# Email (SendGrid)
SENDGRID_API_KEY=SENDGRID_API_KEY_PLACEHOLDER

# Frontend / CDN
PAYMENT_PAGE_BASE_URL=https://your-cloudfront-url.example.com

```

## Backend Environment Variables Reference

| Key                                   | Description                                    |
|---------------------------------------|------------------------------------------------|
| <code>PORT</code>                     | Port on which the backend server runs          |
| <code>BASE_URL</code>                 | Base URL of the backend service                |
| <code>SECRET_KEY</code>               | Secret key for signing tokens or sessions      |
| <code>PHONEPE_BASE_URL</code>         | Base URL for PhonePe sandbox payment API       |
| <code>PHONEPE_API_END_POINT</code>    | PhonePe payment initiation endpoint            |
| <code>MERCHANT_ID</code>              | Merchant identifier provided by PhonePe        |
| <code>MERCHANT_USER_ID</code>         | Merchant user identifier for transactions      |
| <code>SALT_INDEX</code>               | Index used to identify the salt key version    |
| <code>SAMPLE_SALT_KEY</code>          | Salt key used for request signature generation |
| <code>PHONEPE_STATUS_END_POINT</code> | Endpoint to check payment transaction status   |
| <code>RAZORPAY_KEY_ID</code>          | Razorpay public API key for authentication     |

| Key                      | Description                                              |
|--------------------------|----------------------------------------------------------|
| RAZORPAY_TEST_KEY_SECRET | Razorpay secret key for signing requests                 |
| PHONEPE_API_URL_UAT      | PhonePe UAT (testing) API base URL                       |
| PHONEPE_API_URL_PROD     | PhonePe production API base URL                          |
| AES_KEY                  | AES encryption key for data encryption                   |
| PG_HOST                  | PostgreSQL database host address                         |
| PG_PORT                  | PostgreSQL database port                                 |
| PG_USERNAME              | PostgreSQL database username                             |
| PG_PASSWORD              | PostgreSQL database password                             |
| PG_DATABASE              | PostgreSQL database name                                 |
| SUPER_ADMIN_EMAIL        | Email for the super admin account                        |
| SUPER_ADMIN_PASSWORD     | Password for the super admin account                     |
| AWS_REGION               | AWS region where resources are hosted                    |
| AWS_ACCESS_KEY           | AWS access key for API authentication                    |
| AWS_SECRET_KEY           | AWS secret key for API authentication                    |
| S3_BUCKET_NAME           | S3 bucket name for asset storage                         |
| SENDGRID_API_KEY         | API key for sending emails via SendGrid                  |
| PAYMENT_PAGE_BASE_URL    | Base URL for hosted payment page (CloudFront)            |
| ENCRYPTION_SECRET        | Application-wide encryption secret for crypto operations |

## Frontend Dashboard .env

**Location:** `/path/to/dashboard-project/.env` (LOCAL machine only)

 This file is used at BUILD TIME only. It is NOT deployed to S3. The values are embedded in the compiled JavaScript bundle.

```
# Base URL of the backend API and socket
VITE_API_BASE_URL=https://api.your-domain.com/api
VITE_SOCKET_BASE_URL=https://api.your-domain.com
```

## Access in React code

```
const apiUrl = import.meta.env.VITE_API_URL;
```

## Frontend Payment Page .env

**Location:** `/path/to/payment-page-project/.env` (LOCAL machine only)

```
# Base URL of the backend API
VITE_API_BASE_URL=https://api.your-domain.com/api
```

### Access in React code

```
const stripeKey = import.meta.env.VITE_API_BASE_URL;
```

## 9. Route 53 DNS Configuration

| Property       | Value                    |
|----------------|--------------------------|
| Domain         | kg91.pro                 |
| Registrar      | Namecheap                |
| DNS Provider   | AWS Route 53             |
| Hosted Zone ID | (check Route 53 console) |

### Hosted Zone Setup (One-Time)

The hosted zone for kg91.pro is already created in Route 53. Nameservers from Route 53 are configured at Namecheap.

**To view:**

`AWS Console → Route 53 → Hosted Zones → kg91.pro`

The hosted zone contains NS and SOA records. Copy the NS record values (4 nameservers) and ensure they're set at Namecheap.

### Verify nameserver propagation

```
dig NS kg91.pro +short
```

Should show 4 Route 53 nameservers (e.g. `ns-123.awsdns-12.com` )

## Current DNS Records

| Record Name                 | Type      | Value / Target                         | Purpose                 |
|-----------------------------|-----------|----------------------------------------|-------------------------|
| kg91.pro                    | A (Alias) | CloudFront distribution id             | Dashboard (apex domain) |
| api.kingsgate-dev.kg91.pro  | A         | EC2 instance public IP (no Elastic IP) | Backend API             |
| kingsgate-payments.kg91.pro | A (Alias) | CloudFront distribution id             | Payment page            |

## Updating API DNS After EC2 Restart

If EC2 instance is stopped and restarted:

1. **AWS Console** → **EC2** → **Instances** → **Instance** i-00e7d9bb0607e5cc4
2. Copy the new public IPv4 address
3. **Route 53** → **Hosted Zones** → **kg91.pro**
4. Click on the A record for `api.kingsgate-dev.kg91.pro`
5. **Edit** → Update Value to the new IP
6. Save
7. Wait 5 minutes (TTL) for propagation

## Verify

```
dig A api.kingsgate-dev.kg91.pro +short
```

💡 **To avoid this:** Allocate an Elastic IP and attach it to the EC2 instance. Elastic IPs are free while attached to a running instance.

# 10. ACM (AWS Certificate Manager)

ACM manages TLS certificates for CloudFront distributions. Certificates are free, managed by AWS, and auto-renewing.

 **CRITICAL:** CloudFront requires ACM certs in us-east-1 region ONLY. Always switch to us-east-1 before requesting/viewing ACM certs for CloudFront.

## Current ACM Certificates

### Certificate 1

| Property     | Value                               |
|--------------|-------------------------------------|
| Domain       | kg91.pro                            |
| Covers       | kg91.pro (apex only)                |
| Used By      | CloudFront distribution (Dashboard) |
| Region       | us-east-1                           |
| Validation   | DNS (CNAME records in Route 53)     |
| Auto-Renewal | Yes                                 |

### Certificate 2

| Property     | Value                                  |
|--------------|----------------------------------------|
| Domain       | kingsgate-payments.kg91.pro            |
| Used By      | CloudFront distribution (Payment page) |
| Region       | us-east-1                              |
| Validation   | DNS (CNAME records in Route 53)        |
| Auto-Renewal | Yes                                    |

## ACM vs Certbot Comparison

|                 | ACM                              | Certbot (Let's Encrypt) |
|-----------------|----------------------------------|-------------------------|
| <b>Used For</b> | CloudFront distributions         | EC2 Nginx (direct TLS)  |
| <b>Cost</b>     | Free                             | Free                    |
| <b>Renewal</b>  | Automatic (perpetual)            | Manual (90 days)        |
| <b>Region</b>   | Must be us-east-1 for CloudFront | Lives on EC2 instance   |

|                   | ACM                      | Certbot (Let's Encrypt) |
|-------------------|--------------------------|-------------------------|
| <b>Wildcard</b>   | Yes (via DNS validation) | Yes (via DNS challenge) |
| <b>Validation</b> | DNS (CNAME in Route 53)  | HTTP or DNS             |

## For Kingsgate

- **CloudFront** (dashboard, payment page): ACM certs (auto-renewing)
- **EC2 backend API**: Certbot cert (manual renewal every 90 days)

# 11. RDS PostgreSQL

| Property      | Value                                 |
|---------------|---------------------------------------|
| Service       | AWS RDS PostgreSQL                    |
| Database Name | kingsgate_db (or as configured)       |
| Access        | Publicly accessible via credentials   |
| Connections   | EC2 instance (via VPC security group) |

## Connect from Local Machine

Use a GUI tool like pgAdmin, DBeaver, etc. with:

- **Host:** (from .development.env)
- **Port:** (from .development.env)
- **Database:** (from .development.env)
- **Username:** (from .development.env)
- **Password:** (from .development.env)

# 12. Common Troubleshooting

## 1. API returning 502 Bad Gateway

### CHECK:

- `pm2 status` (is nest-app running?)
- `pm2 logs kingsgate-dev --lines 100`

**FIX:**

- `pm2 restart kingsgate-dev`
  - Check `.development.env` file for missing/incorrect values
- 

## 2. API returning 504 Gateway Timeout

**CHECK:**

- Long-running database queries
- External API calls timing out

**FIX:**

- Increase `proxy_read_timeout` in Nginx config
  - Optimize slow queries
- 

## 3. HTTPS certificate error on `api.kingsgate-dev.kg91.pro`

**CHECK:**

- `sudo certbot certificates` (is it expired?)

**FIX:**

```
sudo systemctl stop nginx
sudo certbot renew
sudo systemctl start nginx
```

---

## 4. CloudFront serving old version of SPA

**CHECK:**

- Did you invalidate CloudFront after uploading to S3?

**FIX:**

- CloudFront → Distribution → Invalidations → `/*`
  - Wait for invalidation to complete
- 

## 5. Cannot connect to RDS from EC2

**CHECK:**

- RDS security group inbound rules
- EC2 security group ID in RDS inbound rules

**FIX:**

- RDS → Databases → your-db → VPC security groups
- Edit inbound rule → Port 5432 → Source: EC2 security group ID

---

## 6. PM2 not starting on reboot

**CHECK:**

- `pm2 list` (after ec2 reboot)

**FIX:**

```
pm2 startup # run the command it outputs
pm2 save
```

## 7. Critical Reminders

- No Elastic IP on EC2 - update Route 53 A record if instance restarts
- Certbot renewal is MANUAL - set calendar reminder for every 60 days
- Always invalidate CloudFront after S3 uploads ( `/*` for full deploys)
- integration.js is separate from payment page build - deploy independently
- ACM certs for CloudFront MUST be in us-east-1
- Never delete DNS validation CNAME records for ACM certs
- Backend .env (.development.env) is on EC2 only - never commit to git
- after editing .development.env

```
npm run build
pm2 restart nest-app
```

- Frontend .env is build-time only - values are public in JS bundle